PROJECT SYNOPSIS

on

Blood Donation Network with Smart Matching

An Event-Driven Platform for Finding the Right Donor in Time

Submitted in partial fulfilment of the requirements for the degree of

B.E. B.Tech

AIT-CSE-AIML

Submitted by

Anuj Kumar [24BAI70735]

Shlok Kandola [24BAI70823]

Under the guidance of

Ms. Amandeep Kaur Seekhon



Chandigarh University

[Academic Year: 2026-27]

1. Title

Blood Donation Network with Smart Matching: An Event-Driven Platform for Finding the Right Donor in Time

2. Introduction

Blood cannot be manufactured and has a short shelf life, so hospitals depend on voluntary donors. In emergencies such as accidents, surgeries, or childbirth complications, the difficulty is often not a shortage of donors but the time taken to find a compatible, nearby, and available one. Today this is usually done through phone calls, social media forwards, and WhatsApp groups, which are unstructured and slow, and they reach donors who may be ineligible or far away.

This project proposes a **Blood Donation Network** that connects donors, hospitals, and blood banks on one platform. Its core is a **smart matching service** that ranks donors by blood-group compatibility, distance, urgency, and availability. Emergency requests are handled through a **priority queue**, and donors are alerted through an **event-driven notification system**.

3. Statement of the Problem

- Requesters have no quick way to find compatible donors, so time is lost in manual searching.
- Existing platforms are mostly static donor directories. They do not rank donors by distance or urgency.
- Donors' availability is not tracked. A donor may have donated within the last 90 days, may be travelling, or may be busy at the time.
- Emergency and routine requests are treated alike, so critical cases may wait behind non-critical ones.
- Notifications are manual and unreliable, with no tracking of who was contacted, who responded, and who declined.

The project addresses these gaps by automating donor discovery, ranking, scheduling, and notification.

4. Objectives

- Build a donor and requester registration system with blood group, location, and health eligibility details.
- Implement a matching algorithm service that scores donors on blood-group compatibility, distance (simulated), urgency, and availability.
- Provide donor availability scheduling, including a cooldown period after each donation.
- Implement a priority queue so emergency requests are served before routine ones.
- Build an event-driven notification system that alerts matched donors and escalates if no one responds.

- Provide dashboards for hospitals (request tracking) and donors (history, availability, alerts).
- Test the system with simulated data to evaluate matching speed and quality.

5. Methodology

5.1 System Architecture

The system is built as loosely coupled services communicating through events:

1. **User Service:** registration, login, and profiles for donors, hospitals, and admins.
2. **Request Service:** creates blood requests with blood group, units required, urgency level, and location.
3. **Matching Service:** finds and ranks eligible donors for each request.
4. **Scheduling Service:** manages donor availability slots and cooldown periods.
5. **Notification Service:** consumes events and sends alerts (in-app, email, SMS simulation).
6. **Event Bus:** a message broker or in-memory publisher/subscriber that connects the services.

5.2 Matching Algorithm

Step 1: Blood compatibility filter. Only donors whose blood group is compatible with the recipient are kept, using a compatibility map. For example, O- can donate to all groups, while AB+ can receive from all groups.

Step 2: Eligibility filter. Donors are removed if they are marked unavailable or their last donation was less than the cooldown period ago (typically 90 days for men and 120 days for women, following common guidelines).

Step 3: Distance calculation (simulated). Donor and hospital coordinates are generated within a city grid. The distance is computed with the Haversine formula:

$$d = 2R \cdot \arcsin\left(\sqrt{\sin^2(\Delta\varphi/2) + \cos\varphi_1 \cdot \cos\varphi_2 \cdot \sin^2(\Delta\lambda/2)}\right)$$

where R is the Earth's radius (about 6371 km), φ is latitude, and λ is longitude.

Step 4: Scoring and ranking. Each remaining donor receives a weighted score:

$$Score = w_1 \cdot BloodMatch + w_2 \cdot (1 / Distance) + w_3 \cdot Availability + w_4 \cdot ResponseHistory$$

- BloodMatch is 1.0 for an exact match and a lower value for a compatible but non-identical group.
- The weights (w_1 to w_4) shift with urgency. In critical cases, distance and availability receive higher weights.
- The top N donors are selected for notification.

5.3 Priority Queue for Emergencies

Requests are held in a min-heap or max-heap priority queue ordered by urgency level (Critical > Urgent > Normal). Requests of equal urgency are ordered by waiting time, which prevents starvation of lower-priority requests. Insertion and extraction take O(log n).

Level	Example	Behaviour
Critical	Accident, active bleeding	Immediate matching, wider donor radius, parallel alerts
Urgent	Scheduled surgery within 24 hours	Matched next, sequential alerts
Normal	Routine transfusion	Matched in batches

5.4 Event-Driven Notifications

The system publishes events such as **RequestCreated, DonorMatched, DonorAccepted, DonorDeclined, NoResponseTimeout**, and **DonationCompleted**. Subscribers react independently:

- On **DonorMatched**, the notification service alerts the donor.
- On **DonorDeclined** or **NoResponseTimeout**, the matching service selects the next-best donor and escalates.
- On **DonationCompleted**, the scheduling service starts the donor's cooldown period.

This decoupling makes the system scalable, easy to extend, and fault-tolerant.

5.5 Data and Testing

- **Data:** synthetic datasets of donors (with blood groups distributed in realistic proportions), hospitals, and requests, with simulated coordinates.
- **Testing:** unit tests for compatibility and scoring, integration tests for the event flow, and load tests with thousands of simulated donors.
- **Evaluation metrics:** time to first match, acceptance rate of top-ranked donors, and queue waiting time by urgency level.

6. Requirements

6.1 Software

Component	Options
Language / framework	Python (Flask/FastAPI), Java (Spring Boot), or Node.js
Frontend	HTML, CSS, JavaScript, or React
Database	PostgreSQL or MySQL (MongoDB as an option)
Messaging / events	RabbitMQ, Kafka, Redis Pub/Sub, or an in-memory event bus
Notifications	SMTP email and an SMS/push gateway (simulated for testing)

Component	Options
Tools	Git, Postman, VS Code or IntelliJ, draw.io for diagrams

6.2 Hardware

- Development machine with at least 8 GB RAM, a multi-core processor, and 256 GB storage.
- Internet connection for deployment and testing.

7. Expected Outcomes

- A working prototype that returns a ranked list of suitable donors within seconds of an emergency request.
- Reduced time to locate donors compared with manual methods.
- Emergency requests consistently prioritized over routine ones.
- Automated availability tracking that avoids contacting ineligible donors.
- Reliable, traceable notifications with automatic escalation when donors do not respond.

8. Future Scope

- Integration with real map and routing APIs (Google Maps, OpenStreetMap) for actual travel time instead of simulated distance.
- Mobile apps with GPS-based live location and push notifications.
- Machine learning to predict donor response likelihood and demand for particular blood groups.
- Integration with hospital and blood bank inventory systems and government platforms such as e-RaktKosh.
- Support for platelet, plasma, and organ donor matching.
- Donor rewards and gamification, plus blood donation camp scheduling.
- Security and privacy enhancements, including consent management and encrypted health data.

9. References

[1] World Health Organization, Blood Safety and Availability (fact sheet), who.int.

[2] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, Introduction to Algorithms, MIT Press (priority queues and heaps).

[3] G. Hohpe and B. Woolf, Enterprise Integration Patterns, Addison-Wesley, 2003 (messaging and event-driven design).

[4] S. Newman, Building Microservices, O'Reilly Media (service decomposition and event-based communication).

[5] R. W. Sinnott, "Virtues of the Haversine," Sky and Telescope, vol. 68, no. 2, 1984.

[6] National Blood Transfusion Council (India), Guidelines on Blood Donor Selection and Referral.

[7] e-RaktKosh, Ministry of Health and Family Welfare, Government of India, eraktkosh.in.